

Concepts

Order Book

NautilusTrader provides a high-performance order book implemented in Rust, capable of maintaining full book state from L1 through L3 data. The `OrderBook` is the primary component for tracking public market depth, while the `OwnOrderBook` tracks your own orders separately, enabling filtered views that show true available liquidity.

i This guide documents the Rust API. These types are also available from Python via PyO3 bindings (`nautilus_pyo3.OrderBook` , `nautilus_pyo3.OwnOrderBook`). The v1 legacy Cython `OrderBook` (`nautilus_trader.model.book.OrderBook`) returned by `cache.order_book()` has a similar but not identical interface. Refer to the API reference for differences.

Book types

`OrderBook` instances are maintained per instrument for both backtesting and live trading:

- `L3_MBO` : Market by order data. Tracks every order at every price level, keyed by order ID.
- `L2_MBP` : Market by price data. Aggregates orders by price level (one entry per price).
- `L1_MBP` : Top-of-book data, also known as best bid and offer (BBO). Captures only the best prices.

i Top-of-book data such as `QuoteTick` , `TradeTick` and `Bar` can also maintain `L1_MBP` books.

Subscribing to book data

Strategies and actors subscribe to order book updates through the following methods. Subscriptions and handlers are part of the Python strategy/actor layer:

```
# L3/L2 incremental deltas
self.subscribe_order_book_deltas(instrument_id)
```



```
# Aggregated depth snapshots (up to 10 levels)
self.subscribe_order_book_depth(instrument_id)

# Full book snapshots at a timed interval
self.subscribe_order_book_at_interval(instrument_id, interval_ms=1000)
```

Each subscription type delivers data to the corresponding handler:

```
def on_order_book_deltas(self, deltas: OrderBookDeltas) -> None:
    ...

def on_order_book_depth(self, depth: OrderBookDepth10) -> None:
    ...

def on_order_book(self, order_book: OrderBook) -> None:
    ...
```

Accessing the book

The `OrderBook` exposes top-of-book accessors:

```
let best_bid: Option<Price> = book.best_bid_price();
let best_ask: Option<Price> = book.best_ask_price();
let spread: Option<f64> = book.spread();
let midpoint: Option<f64> = book.midpoint();
```

Analysis methods

The `OrderBook` supports market depth analysis and execution simulation:

```
// Average fill price for a given quantity
let avg_px = book.get_avg_px_for_quantity(quantity, OrderSide::Buy);

// Average price and quantity for a target exposure (notional)
let (price, qty, exposure) =
    book.get_avg_px_qty_for_exposure(target_exposure, OrderSide::Buy);

// Cumulative quantity available at or better than a price
let qty = book.get_quantity_for_price(price, OrderSide::Buy);

// Quantity at a specific price level only
let qty = book.get_quantity_at_level(price, OrderSide::Buy, 2);
```

```
// Simulate fills against the book
let fills: Vec<(Price, Quantity)> = book.simulate_fills(&order);

// All crossed levels regardless of order quantity
let levels = book.get_all_crossed_levels(OrderSide::Buy, price, 2);
```

Integrity checks

The `book_check_integrity` function validates that the book state is consistent with its type:

- **L1_MBP**: No more than one level per side.
- **L2_MBP**: No more than one order per price level.
- **L3_MBO**: No structural constraints (any number of orders at any level).
- **All types**: Best bid must not exceed best ask (crossed book). Locked markets (bid == ask) are considered valid.

These checks run internally during delta application. The instrument ID of incoming deltas is also validated against the book's instrument ID, returning

`BookIntegrityError::InstrumentMismatch` on mismatch.

Pretty printing

Both `OrderBook` and `OwnOrderBook` provide a `pprint` method that renders the book as a human-readable table:

```
book.pprint(5, None);
book.pprint(5, Some(Decimal::new(1, 2))); // group_size = 0.01
```



The `group_size` parameter buckets price levels into coarser groups for instruments with fine tick sizes. The output is a formatted table with bids on the left, prices in the center, and asks on the right.

Own order book

The `OwnOrderBook` tracks your own working orders separately from the public book. Market making and other quoting strategies use it to estimate available liquidity at each price level after subtracting their own orders.

Execution engines maintain own books when `manage_own_order_books` is enabled. The cache updates an existing own book as order events change state. Eligible orders have a price and do not use `IOC` or `FOK` time in force. Terminal events may still clean up an existing own book entry, even when the order would not otherwise be eligible for tracking.

Order lifecycle

The `OwnOrderBook` tracks orders through their lifecycle. Orders are added when submitted or materialized from reconciliation, updated as state changes arrive, and removed when they close. Updates include accepted, pending update, pending cancel, partially filled, filled, canceled, expired, rejected, and denied states as supported by the order model.

Each `OwnBookOrder` carries:

- `client_order_id`: Client order ID used to reconcile the own book with cache state.
- `venue_order_id`: Venue order ID when one has been assigned.
- `side`, `price`, and `size`: Order side and the remaining own-book price level.
- `order_type` and `time_in_force`: Order type metadata used by filters and diagnostics.
- `status`: Current order status, such as `SUBMITTED`, `ACCEPTED`, or `PENDING_CANCEL`.
- `ts_last`: Timestamp of the latest order event applied to this own-book order.
- `ts_accepted`: Timestamp when the order was accepted by the venue.
- `ts_submitted`: Timestamp when the order was submitted.
- `ts_init`: Timestamp when the order was initialized.

These fields let filtered views include or exclude own orders by status and acceptance time (see [Status and time filtering](#)).

Auditing

The `audit_open_orders` method reconciles an own book against a set of valid client order IDs. Any own-book order not in the provided set is removed and logged as an audit error.

`Cache::audit_own_order_books` builds this set from open and in-flight orders so submitted orders are not removed during normal venue latency windows. Live systems can run this audit periodically through the own-books audit interval.

Querying

```
// Check if a specific order is tracked
let in_book = own_book.is_order_in_book(&client_order_id);

// Get all tracked order IDs per side
let bid_ids = own_book.bid_client_order_ids();
let ask_ids = own_book.ask_client_order_ids();

// Aggregated quantities per price level
let bid_qty = own_book.bid_quantity(None, None, None, None, None);
let ask_qty = own_book.ask_quantity(None, None, None, None, None);

// Pretty print
own_book.pprint(5, None);
```

Filtered views

Subtract your own orders from the public book to see net available liquidity:

```
// Filtered maps of price -> quantity (own orders subtracted)
let net_bids = book.bids_filtered_as_map(Some(10), Some(&own_book), None, None, None);
let net_asks = book.asks_filtered_as_map(Some(10), Some(&own_book), None, None, None);

// Full filtered OrderBook with all analysis methods available
let filtered = book.filtered_view(Some(&own_book), Some(10), None, None, None);
let avg_px = filtered.get_avg_px_for_quantity(quantity, OrderSide::Buy);
```

The `filtered_view` method returns a new `OrderBook` with your own sizes subtracted, giving access to the full set of analysis methods (`spread`, `midpoint`, `get_avg_px_for_quantity`, etc.) on the net book.

Status and time filtering

Filtered views support optional status and time-based filtering for own orders:

```
let status = Some(AHashSet::from([OrderStatus::Accepted]));

// Only subtract ACCEPTED orders (ignore SUBMITTED, PENDING_CANCEL, etc.)
```

```
let filtered = book.filtered_view(Some(&own_book), None, status, None, None);
```

The `accepted_buffer_ns` parameter provides a grace period: when set, only orders where `ts_accepted + buffer <= now` are included. This excludes recently accepted orders that may not yet appear in the public book feed. The buffer applies to the `ts_accepted` field regardless of order status. Combine with a status filter to also exclude non-accepted orders.

```
// Only subtract orders accepted at least 500ms ago
let filtered = book.filtered_view(
    Some(&own_book),
    None,
    None,
    Some(500_000_000),
    Some(clock.timestamp_ns()),
);
```

Binary markets

For binary/prediction markets (e.g., Polymarket), instruments have two complementary sides (YES and NO) where prices sum to 1.0. A bid on the NO side at 0.40 is economically equivalent to an ask on the YES side at 0.60.

The `OwnOrderBook::combined_with_opposite` method handles this transformation, merging your orders from both sides into a single view:

```
let yes_own = own_yes_book
    .cloned()
    .unwrap_or_else(|| OwnOrderBook::new(yes_instrument_id));

let no_own = own_no_book
    .cloned()
    .unwrap_or_else(|| OwnOrderBook::new(no_instrument_id));

// Merge NO-side orders with parity price transform (1 - price)
let combined = yes_own.combined_with_opposite(&no_own).unwrap();

// Filter the public YES book using the combined own book
let filtered = book.filtered_view(Some(&combined), None, None, None, None);
```

The transformation works as follows:

- NO asks at price P become bids at price 1 - P in the combined book.

- NO bids at price P become asks at price $1 - P$ in the combined book.

This gives a complete picture of your own liquidity across both sides of the market.

< Custom Data

Previous Page

Execution >

Next Page